

Polyglot AI Team OS v2

终端优先、本地 Agent 优先、可随时转向的低 Token AI 团队运行系统

这一版不是把旧方案和新方案机械拼接，而是取其精华、去其糟粕：保留你原来最有产品感的动态组队、本地 CLI Agent、飞书协同、可中途转向，再融合低 Token 协调与结构化运行时。

核心参考：你原始的 PDD / MAD 思路，以及网页检索得到的 ClawSwarm、Multica、Mission Control、层级化多智能体设计模式。

1. 这一版保留什么，砍掉什么

这次的目标不是继续堆抽象，而是把最值钱的东西留下来：

- 保留本地现成 Agent 可直接复用的优势
- 保留 AI 自动搭团队、自动拆任务的 Meta-Workflow 核心
- 保留执行过程中人类可随时插话改方向的“活性”
- 保留飞书 / IM 作为真实协同入口
- 保留低 Token、结构化协同、成本可控的收敛原则

砍掉的是平台感过重、配置感过重、自由群聊过重和第一版就大而全的冲动。

不是做一个会聊天的 Agent 平台，而是做一个可被人随时打断、随时转向、能复用本地 Agent 劳动力的 AI Team 运行系统。

2. 原始想法里真正该保留的精华

2.1 本地 CLI Agent 优先

系统要自动发现并直接复用户已安装的 Claude Code、Codex、Hermes、OpenClaw、OpenCode 等 CLI Agent，把它们当“工人”纳入统一调度，尽量不强制用户额外接一堆云端配置。

2.2 终端优先，不先做重 Web 平台

第一入口是 CLI、Terminal、本地工作目录和本地代码环境。Web 更适合作为只读监控面板，而不是唯一工作面。

2.3 AI 自动搭 Workflow，而不是人手写死 Stage

Workflow 本身是 AI 动态生成的中间组织结构，而不是用户提前写好的静态 YAML。系统应根据任务复杂度决定怎么拆、叫几个工人、用什么工具。

2.4 任务执行过程中，人可以随时改方向

系统必须天然支持 interrupt、steer、pause、resume、local re-plan、partial reroute。人类中途插话不是异常，而是产品正常工作流的一部分。

2.5 飞书 / IM 是真实入口

飞书不是通知渠道，而是一等协同界面。用户可以在飞书里发任务、看 blocker、审批关键动作、接收阶段报告并中途改方向。

3. 原来方案里的糟粕，必须砍掉

- 平台感过重：太像基础设施蓝图，不像第一版就值得用的产品。
- 声明式 Workflow 感过重：YAML、DAG、schema 可以留在内核，不该成为前台卖点。
- 群聊式协作过重：看起来热闹，但高成本、低效率、容易浪费 Token。
- 过早做大而全 Web 平台：可视化编辑器、复杂权限系统、重型插件市场都不该抢前排。

4. 融合后的新产品定义

产品名：Polyglot AI Team OS

一句话定义：一个终端优先、本地 Agent 优先、飞书可协同、可随时转向、低 Token 协调的 AI 团队运行系统。

它不是：多 Agent 聊天室、静态 DAG 引擎、传统任务板套 AI 壳、必须先配满云端环境的平台。

它是：自动组织本地与远端 Agent 劳动力的运行时；能把目标动态拆成任务与角色分工的元编排系统；能让人类在运行中随时插话修正方向的 Team OS。

5. 这版必须坚持的五个主张

- **Local-first Agent Workforce**：优先复用本地 Claude Code、Codex、Hermes、OpenClaw、OpenCode 等现成执行体。
- **Meta-Workflow**：用户给的是目标，不是完整流程；系统动态生成 Team Plan / Task Graph，并在失败时重规划。
- **Human Steering is First-Class**：用户可以随时打断、改方向、加约束、换角色。
- **Feishu / IM is a Real Interface**：飞书里能发起任务、看进度、回应 blocker、审批关键节点、接收结果。
- **Use Systems for Coordination, Not Conversations**：协作要像系统，不像会议；状态流转优先于长对话。

6. 新版架构：只保留真正有用的层次

- **Layer 1: Interfaces** —— Terminal CLI（主入口）、Feishu / IM（协同入口）、Web Dashboard（只读监控，可后置）
- **Layer 2: Meta Planner** —— 理解目标、估计复杂度、生成 Team Plan / Task Graph、失败 re-plan、吸收中途转向
- **Layer 3: Runtime Orchestrator** —— 启动 Worker、局部上下文注入、工具白名单、产物与状态更新、结果验证
- **Layer 4: Worker Fabric** —— 统一封装本地 CLI Worker 和 API Worker，管理工作目录与执行环境
- **Layer 5: Skill / Artifact / Metrics Layer** —— skill、artifact、event log、metrics、token / latency / cost tracking、replay

7. 核心执行流

- **Step 1**：用户给目标，可以是一句话、PRD、链接、飞书消息或终端命令。
- **Step 2**：Meta Planner 生成结构化 Team Plan：task_name、complexity_level、execution_mode、roles_needed、issue_graph、budget、approval_points。
- **Step 3**：Runtime 只实例化当前需要的执行体，不默认把所有 Agent 拉起来群聊。
- **Step 4**：每个 Worker 只拿当前任务目标、必要的上游摘要、关键文件和预算，不拿全量历史。
- **Step 5**：Worker 结构化回报 status、artifact、summary、blocker、next_action、cost。
- **Step 6**：用户可在任意阶段通过终端或飞书插话，系统做局部 re-plan，尽量不整局重跑。

8. Token 成本怎么控制

保留上一版关于低成本协调的精华，但不让它压过产品灵魂：

- 默认单 Agent，简单任务先单跑
- 只有并行有收益时才扩成多 Agent
- 结构化交接代替自由群聊
- 强模型只做初始拆解、复杂重规划、最终验收
- 技能与摘要强缓存，让系统越用越省
- 每个阶段都有 token、latency、retry budget，超限就自动降级

9. 值得沉淀成长期护城河的能力

- Local Agent Discovery
- Dynamic Team Formation
- Interruptible Runtime
- Feishu-native Collaboration
- Structured Task Board
- Skill Compounding

这六类能力比花哨 UI 更能构成长期价值。

10. MVP 应该怎么切

第一版必须有

- CLI 主入口
- 本地 CLI Agent 自动探测
- Meta Planner
- 基础 Task Board
- Runtime Orchestrator
- 局部 re-plan
- Feishu 消息接入（至少发任务、收结果、改方向）
- cost / token tracking
- artifact + skill store

第一版不要做

- 复杂 Web 可视化编辑器
- 大群聊房间式 UI
- 很重的多租户权限系统
- 远程集群式 worker 编排
- 大而全插件市场

第一版最该证明的是：用户已经装好的 AI Agent，可以被系统接起来，像一支可转向的小团队一样干活。

11. 用户最终看到的体验

场景 **A**：终端里发起 —— 用户输入“做一个带 JWT 的后台 API，并且把测试跑通”，系统自动判断复杂度、决定单 Agent / 多 Agent、调用本地 Claude/Codex 执行、失败时局部重规划，最后给出代码、测试结果、成本摘要。

场景 **B**：飞书里中途改方向 —— 用户在飞书里说“别继续做商品模块，先把登录打通”或“先别写代码，给我个方案”，系统接收后局部调整 Team Plan、暂停不必要 Worker、重新派工并继续把结果回推到飞书。

这个体验才真正像“带着一支 AI team 干活”。

12. 竞品对比：你 vs ClawSwarm vs Multica

维度	你的方向：Polyglot AI Team OS	ClawSwarm	Multica
核心定位	可转向、可复用、低 Token 的 AI Team Runtime	OpenClaw 的多 Agent 群聊协作层	Managed agents 平台，把 Agent 当正式团队成员
主要入口	CLI / Terminal + Feishu / IM	Web 群聊界面 + OpenClaw 插件	Web 平台 + daemon/runtime 件
Agent 来源	本地 CLI agent 优先，自动发现 Claude/Codex/Hermes/OpenClaw 等	主要围绕 OpenClaw 生态	多种 agent CLI/provider，偏统一平台接入
编排方式	Meta-Workflow，AI 动态生成 team plan / task graph	多 Agent 在统一群聊里协作	issue lifecycle + squads + runtimes
人类中途改方向	强，一等能力；可 interrupt / steer / local re-plan	相对弱，核心叙事更偏 agent 彼此对话	有任务管理，但“中途转向”不是最突出的主卖点
协作媒介	结构化任务板 + 必要短消息 + 事件流	群聊 / 对话流	任务板 / issue / status / conversation
Token 控制思路	默认单 Agent、局部上下文、结构化交接、预算控制	更容易因群聊式协作带来高通信成本	有 managed runtime，但主叙事不是 low-token orchestration
飞书协同	强，IM 是真实入口和干预面	不是核心卖点	不是核心卖点
最适合的场景	个人开发者、小团队、本地 agent 劳动力编排、可随时转向任务	想要展示 swarm 群体协作感的 OpenClaw 用户	想把 agent 正式纳入任务系统的团队
真正差异化	动态建队 + 本地 agent 优先 + 人类随时改方向 + 低 Token 协调	多 Agent 群聊体验强	managed agents、squads、task lifecycle 强

ClawSwarm 解决的是“让多个 Agent 一起聊”；Multica 解决的是“让 Agent 像正式团队成员一样被管理”；你的方向要解决的是“让现有 Agent 劳动力被动态组织、被人类随时打断修正，并以更低成本持续产出”。

13. 对外怎么讲

- 对比普通单 **Agent**：你不只是给一次性回答，而是在组织任务、调执行体、接收中途转向、沉淀经验、控制成本。
- 对比 **ClawSwarm**：你保留群体协作价值，但不把自由群聊作为默认协同方式。
- 对比 **Multica**：你保留 managed agents 的任务生命周期，但更强调本地 Agent 优先、动态 Team Formation、Human Steering、IM 协同入口。

这不是一个 AI 聊天产品，也不是一个死板的 Workflow 引擎，而是一个可转向、可复用、可低成本运行的 AI Team Runtime。

14. 最终结论

融合之后，真正应该留下来的核心是这五件事：

- 本地 CLI Agent 优先
- Meta-Workflow 动态建队与拆任务
- 人类中途可插话改方向
- 飞书 / IM 作为真实协同入口
- 用结构化系统协调替代高成本群聊

真正该砍掉的是：过度平台感、过重配置感、过重群聊感、过早做大而全 Web 平台。

真正有价值的，不是让更多 Agent 一起说话，而是让现有 Agent 劳动力被更聪明地组织、被人随时打断修正、并且越跑越省。